SocketCAN

The CAN protocol is supported in the Linux environment through SocketCAN, which is a set of drivers and a networking stack. Volkswagen Research contributed the initial code by adding support to the Linux kernel v2.6.25 in 2008. Current kernel versions come with a rich set of drivers and networking stack support for a wide variety of CAN interfaces.

Setting up Virtual CAN

Are you thinking of playing around with SocketCAN? The Virtual CAN (vcan) interface provided by the Linux kernel is a good solution for this. It allows initial application development based on vcan and later migration to physical interfaces supported by real hardware. It works almost like a loopback interface in a TCP/IP stack, where any frames written to vcan can be captured on the same machine.

The following simple steps will help you to start with SocketCAN.

To load the kernel module for virtual can support, give the following command:

```
sudo modprobe vcan
```

Then add a virtual interface as follows:

```
ip link add dev vcan0 type vcan
```

Next, configure and bring up the interface. For a physical interface, we can set the bit rate (speed of the CANBus) and communication mode like loopback or listen only, using the following command:

```
ip link set up vcan0
```

To list enabled interfaces, type:

```
ifconfig -a
ifconfig vcan0
```

Working with user space utilities

For user space communication, the *can-utils* package comes with various commands. Here are some of the steps for setting up *can-utils*, and a few examples of its basic usage.

In the Ubuntu Linux distribution, *can-utils* is available through the software repository:

```
sudo apt-get install can-utils
```

If *can-utils* is not available from the package manager, we can build it from the source code, as follows. This is helpful when cross-compiling is required for target platforms.

```
git clone
can-utils
make
make install
```

The following are some examples of the usage of *can-utils*. We can replace *vcan0* by *can0*, *can1*, etc, when physical interfaces are available.

a. To send a standard *can* frame with 0x101 as the ID and 0x41, 0x42, 0x43, 0x44 as a 4-byte payload, type:

```
vcan0 101#41424344
```

b. To send an extended frame with 0xA1B2 as the ID, give the following command:

```
cansend vcan0 0000A1B2#3450
```

c. To capture all incoming *can* frames on *vcan0* along with timestamps, type:

```
candump -t Absolute vcan0
```

d. Interface name can be *any* to capture all the interfaces, as follows:

```
candump -t A any
```

e. To store all captured frames in a log file, use the following command:

```
candump -l vcan0
```

These recorded frames can be replayed using the player or be displayed in a user friendly format using *log2asc*.

cangen is used to generate random frames continuously, so do explore the various options of *cangen* to control IDs, the payload and the timing of generated frames.

```
cangen vcan0
```

C/C++ APIs for application development

In C/C++, CAN APIs are supported by extending Berkeley Socket APIs and by adding a new protocol family called *PF_CAN*. Let's look at some simple steps for developing a CANBus application using these APIs.

Step 1: Create an empty socket API with *PF_CAN* as the protocol family:

```
int s = socket(PF_CAN, SOCK_RAW, CAN_RAW);
```

Step 2: Bind the socket to one of the interfaces (*vcan0* or *canx*):